Лабораторная работа №3

«Базовые растровые алгоритмы»

Дисциплина: Компьютерная графика
Выполнил: студент Никончик Виктор Александрович

курс 3, группа 3

1 Цель работы

- Изучить базовые алгоритмы растеризации отрезков и кривых: пошаговый алгоритм, алгоритм ЦДА, алгоритм Брезенхема для отрезка и алгоритм Брезенхема для окружности.
- Создать десктопное приложение, иллюстрирующее работу этих алгоритмов на дискретной сетке с системой координат, осями, линиями сетки, подписями и масштабированием, и оценить время работы реализованных алгоритмов.

2 Математические основы методов

2.1 Задача растеризации

Растровое устройство вывода представляет изображение в виде прямоугольной сетки пикселей. Растеризация — выбор множества пикселей, наилучшим образом приближающего непрерывную фигуру. Для отрезка требуется, чтобы растровая линия была связной, не имела лишней толщины и строилась быстро.

В программе пиксель с целочисленными координатами (x, y) — это клетка сетки с центром в точке (x, y); её границы проходят по линиям $x \pm 0{,}5$ и $y \pm 0{,}5$. Ось y направлена вверх. Поэтому концы отрезка и центр окружности, заданные целыми числами, совпадают с центрами клеток.

Все алгоритмы для отрезков с концами (x_1, y_1) и (x_2, y_2) используют одну идею: если отрезок пологий ($|\Delta x| \geq |\Delta y|$, где $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$), координата x меняется с шагом 1 и для каждого x выбирается ровно один y; если крутой — роли осей меняются. Так получается связная линия толщиной в один пиксель.

2.2 Пошаговый алгоритм

Используется уравнение прямой. Для пологого отрезка вычисляются коэффициенты

$$k = \frac{\Delta y}{\Delta x}, \qquad b = y_1 - k\,x_1$$

и для каждого целого x от x_1 до x_2 закрашивается пиксель $(x, \mathrm{round}(kx + b))$. Для крутого отрезка аналогично $x = ky + b$ с $k = \Delta x / \Delta y$. На каждом шаге выполняются вещественное умножение, сложение и округление.

2.3 Алгоритм ЦДА

Цифровой дифференциальный анализатор заменяет умножение сложением: при равномерном шаге по одной оси приращение по другой постоянно. Число шагов и приращения равны

$$L = max\left(|\Delta x|,\ |\Delta y|\right), \qquad \delta x = \frac{\Delta x}{L}, \qquad \delta y = \frac{\Delta y}{L}$$

Начиная с точки (x_1, y_1), выполняется $L + 1$ шаг: закрашивается пиксель (round(x), round(y)), затем x += δx, y += δy. Одно из приращений равно ±1, поэтому алгоритм автоматически выбирает основную ось. Вычисления остаются вещественными, и на длинных отрезках накапливается ошибка округления.

2.4 Алгоритм Брезенхема для отрезка

Алгоритм использует только целочисленную арифметику. Для пологого отрезка на каждом шаге по x координата y либо сохраняется, либо меняется на 1; выбор определяется тем, по какую сторону от середины между двумя кандидатами проходит идеальная прямая. Это расстояние, умноженное на $2\Delta x$ для избавления от дробей, хранится в целочисленной ошибке e:

$$e_0 = 2|\Delta y| - |\Delta x|$$

- закрашивается текущий пиксель (x, y);
- если $e \geq 0$, выполняется шаг по y и e −= $2|\Delta x|$;
- затем e += $2|\Delta y|$ и выполняется шаг по x.

Для крутых отрезков роли x и y меняются, направление шагов задаётся знаками Δx и Δy. На каждой итерации — только сложения, вычитания и сравнение с нулём.

2.5 Алгоритм Брезенхема для окружности

Окружность радиуса R с центром (xc, yc) симметрична относительно осей и диагоналей, поэтому вычисляется только 1/8 часть — дуга от точки (0, R) до диагонали $x = y$. Каждая найденная точка (x, y) даёт 8 пикселей: (xc ± x, yc ± y) и (xc ± y, yc ± x).

На этой дуге x на каждом шаге увеличивается на 1, а y либо сохраняется, либо уменьшается на 1. Выбор делается по целочисленной величине d, которая получается подстановкой кандидатов в функцию $x^2 + y^2 - R^2$:

$$d_0 = 3 - 2R$$

- если $d < 0$, y не меняется и d += $4x + 6$;
- если $d \geq 0$, y −= 1 и d += $4(x - y) + 10$;
- затем x += 1; цикл продолжается, пока $x \leq y$.

Приращения d выводятся как разности значений функции в соседних точках, поэтому квадраты сокращаются и остаются только целочисленные сложения.

3 Параметры окружения и используемый стек

- Язык и стандарт: Java 21
- Графический фреймворк: Java Swing / AWT (Graphics2D)
- Система сборки: Maven
- Целевая платформа: любая ОС с JRE 21

4 Интерфейс и реализация приложения

Главное окно разделено на две части. Слева — область построения: дискретная сетка, оси x и y со стрелками и подписями значений. Шаг подписей подбирается автоматически в зависимости от масштаба. Растровые пиксели закрашиваются синим, идеальная фигура (отрезок или окружность) рисуется поверх красной линией, что позволяет сравнить её с растровым приближением.

Справа — панель управления: выбор алгоритма, параметры фигуры (x_1, y_1, x_2, y_2 для отрезка или xc, yc, R для окружности), кнопки «Построить» и «Очистить». В блоке «Вид» задаётся масштаб (размер пикселя сетки в экранных точках), а кнопка возвращает начало координат в центр области. Там же приведено пояснение привязки целочисленных координат к сетке. Масштаб также меняется колесом мыши относительно курсора, а перетаскивание мышью сдвигает изображение. В блоке «Результат» выводятся число закрашенных пикселей и время работы алгоритма, а в строке состояния — координаты пикселя под курсором.

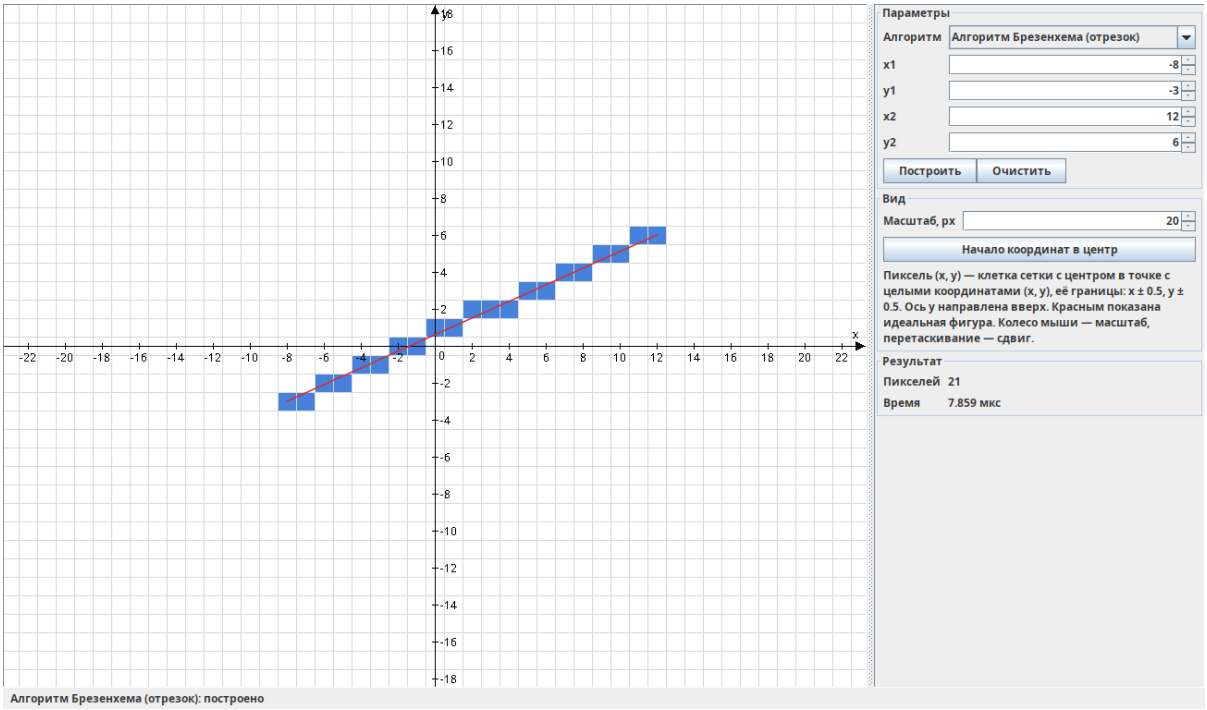


Рис. 1: Отрезок (−8, −3) – (12, 6), алгоритм Брезенхема

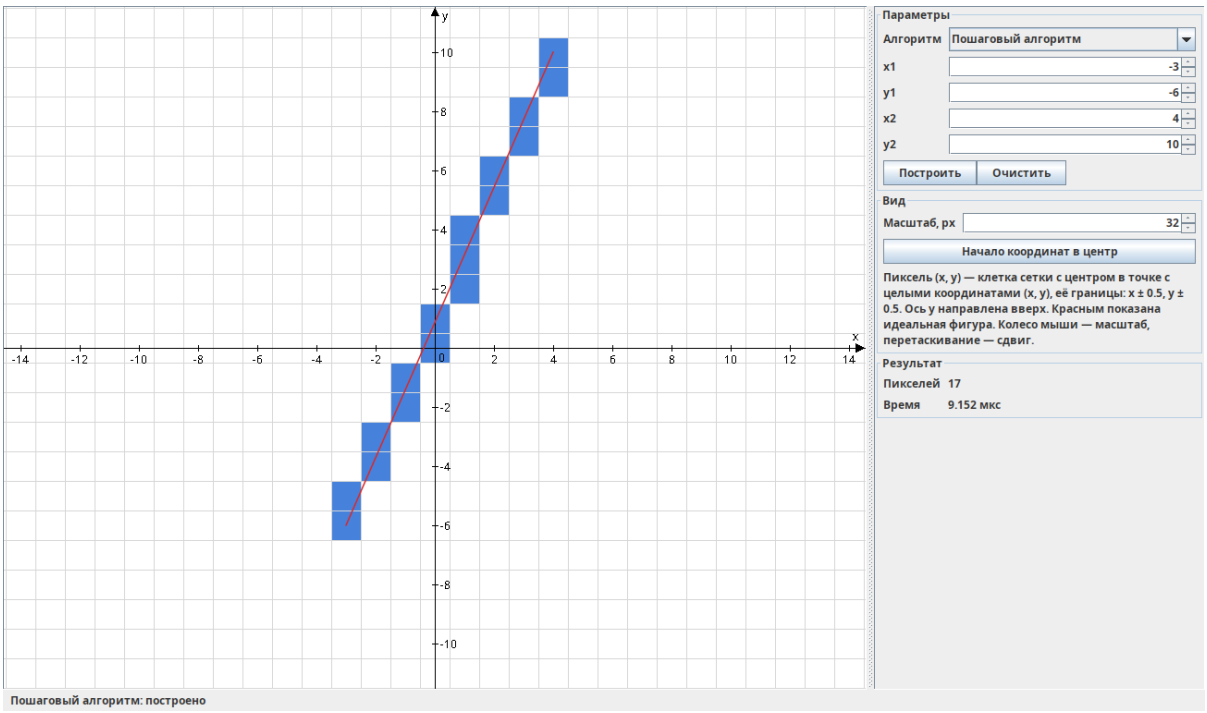


Рис. 2: Крутой отрезок (−3, −6) – (4, 10), пошаговый алгоритм (шаг по оси y)

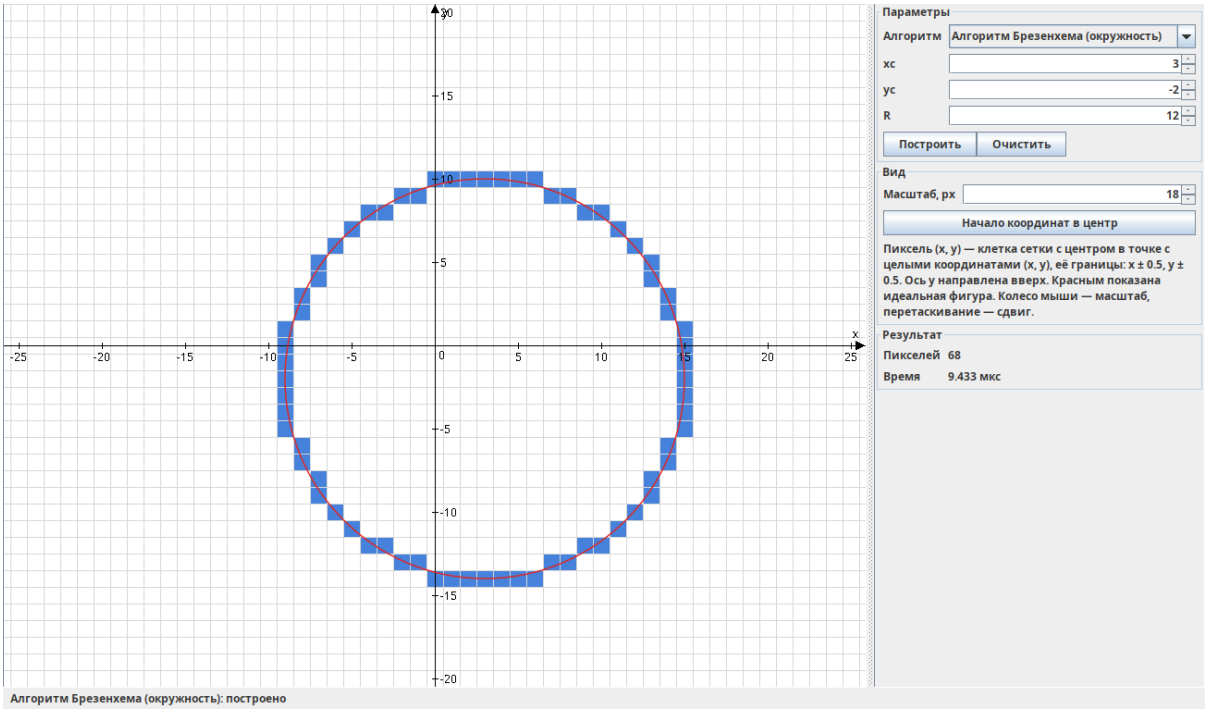


Рис. 3: Окружность с центром (3, −2) и R = 12, алгоритм Брезенхема

Код разделён на два пакета:

- core — алгоритмы, не зависящие от интерфейса: Rasterizer (четыре алгоритма растеризации), Algorithm (перечисление алгоритмов), PixelSink (интерфейс получателя пикселей);
- ui — интерфейс: MainFrame (главное окно), GridPanel (сетка, оси, подписи, масштаб), Form.

Алгоритмы передают пиксели через интерфейс PixelSink, поэтому одна и та же реализация используется и для отрисовки, и для замера времени.

5 Пример работы алгоритмов

Для отрезка (0, 0) – (7, 3): $\Delta x = 7$, $\Delta y = 3$, отрезок пологий. Пошаговый алгоритм: $k = 3/7 \approx 0{,}4286$, $b = 0$. ЦДА: $L = 7$, $\delta x = 1$, $\delta y \approx 0{,}4286$. Брезенхем: $e_0 = 2 \cdot 3 - 7 = -1$, при $e \geq 0$ e −= 14, на каждом шаге e += 6.

Таблица 1: Вычисления для отрезка (0, 0) – (7, 3)

x	Пошаговый: kx + b	ЦДА: y	Брезенхем: e	Пиксель
0	0,0000	0,0000	−1	(0, 0)
1	0,4286	0,4286	5	(1, 0)
2	0,8571	0,8571	−3	(2, 1)
3	1,2857	1,2857	3	(3, 1)
4	1,7143	1,7143	−5	(4, 2)
5	2,1429	2,1429	1	(5, 2)
6	2,5714	2,5714	−7	(6, 3)
7	3,0000	3,0000	−1	(7, 3)

Все три алгоритма дают одинаковый набор пикселей. В пошаговом алгоритме и ЦДА округляется вещественное значение y; в алгоритме Брезенхема шаг по y делается, когда $e \geq 0$ (на шагах 1, 3, 5), что соответствует переходу y через середину клетки.

Для окружности с центром (0, 0) и $R = 5$: $d_0 = 3 - 10 = -7$.

Таблица 2: Вычисления для окружности R = 5

x	y	d	Решение	Новое d
0	5	−7	d < 0: d += 4·0 + 6	−1
1	5	−1	d < 0: d += 4·1 + 6	9
2	5	9	d ≥ 0: d += 4(2 − 5) + 10, y = 4	7
3	4	7	d ≥ 0: d += 4(3 − 4) + 10, y = 3	13

На шаге $x = 4$ условие $x \leq y$ нарушается, и цикл завершается. Точки (0, 5), (1, 5), (2, 5), (3, 4) с отражением в 8 октантов образуют окружность из 28 различных пикселей.

6 Временные характеристики

Время измерялось для реализованных алгоритмов (без отрисовки) с помощью System.nanoTime(): после прогрева кода алгоритм многократно запускался в течение 1 с, и полное время делилось на число запусков.

Таблица 3: Среднее время одного запуска, мкс

Фигура	Пикселей	Пошаговый	ЦДА	Брезенхем
Отрезок (0, 0) – (10, 4)	11	0,032	0,042	0,027
Отрезок (0, 0) – (100, 37)	101	0,173	0,231	0,072
Отрезок (0, 0) – (1000, 370)	1001	1,488	2,078	0,543

Таблица 4: Алгоритм Брезенхема для окружности, мкс

Радиус R	Итераций	Время
10	8	0,042
100	71	0,250
1000	708	2,375

Время всех алгоритмов растёт линейно с числом пикселей. Алгоритм Брезенхема для отрезка быстрее остальных в 2,5–4 раза, так как использует только целочисленные операции. ЦДА в данной реализации оказался медленнее пошагового алгоритма: экономия на умножении меньше, чем затраты на два округления вещественных чисел (round(x) и round(y)) на каждом шаге вместо одного. Алгоритм для окружности на каждой итерации ставит 8 пикселей, поэтому время одной итерации больше, чем у отрезка.

В самой программе выводится время однократного запуска.

7 Выводы

В ходе выполнения работы было разработано десктопное приложение, иллюстрирующее работу базовых растровых алгоритмов, а также:

- реализованы пошаговый алгоритм, алгоритм ЦДА и алгоритм Брезенхема для отрезков, алгоритм Брезенхема для окружности;
- реализован вывод дискретной сетки с системой координат, осями, подписями и масштабированием; пиксель (x, y) привязан к клетке с центром в целочисленной точке (x, y);
- на примере показано, что алгоритмы для отрезков дают одинаковый результат, но различаются вычислительной сложностью шага;
- измерено время работы: целочисленный алгоритм Брезенхема оказался самым быстрым, алгоритмы с вещественной арифметикой медленнее в 2,5–4 раза.